07

Design a Vending Machine

In this chapter, we will explore the design of a vending machine system that allows users to select and purchase products, dispense items, manage inventory, and process payments. Although real-world vending machines involve hardware components, like coin dispensers, card readers, and touchscreens, we'll focus on modeling the system's states, data, and core functionality.

Vending Machine

Requirements Gathering

Here is an example of a typical prompt an interviewer might give:

"Imagine you're at a vending machine, craving a snack. You insert some cash, select your favorite item, and within seconds, it drops into the tray. The machine also gives you the right change if needed. Behind the scenes, the system is working smoothly to track inventory, handle payments, and make sure everything runs efficiently. Now, let's design a vending machine that does all this."

Requirements clarification

Here is an example of how a conversation between a candidate and an interviewer might unfold:

Candidate: Does the vending machine support different types of products?
Interviewer: Yes, the vending machine supports a variety of products, such as snacks, beverages, and other items.

Candidate: How are products organized within the vending machine? Are they placed in specific racks or arranged differently? Also, I assume each product needs a unique identifier, like a product code, along with attributes such as its price.
Interviewer: Yes, products are placed in specific racks, with each rack holding only one type of product at a time. Each product also has a unique product code and a price tag.

Candidate: How will payments be processed in the vending machine?
Interviewer: The vending machine should only accept cash payments and calculate change if needed.

Candidate: How does the vending machine handle cases where a user selects a product that is out of stock or unavailable?
Interviewer: In such cases, the system should be able to check if a product is available. If not, it should display an error message to the user.

Candidate: If a user inserts money less than the product's full price, can they add more incrementally?
Interviewer: For this design, let's assume users insert the full amount in one step. If the inserted amount is insufficient, the vending machine should return the money and display an error.

Candidate: Are there any restrictions on who can access the vending machine?
Interviewer: Access to the vending machine is available to users and admins, with different privileges. Users should be able to select and purchase products by specifying the product code. Admins, however, are responsible for adding or removing products from the machine.

Candidate: Are there any security or inventory tracking requirements for the vending machine?
Interviewer: Yes. The vending machine should track inventory, and only the admin can add or remove products.

Requirements

Here are the functional requirements based on the conversation:

- **Product selection:** Users should be able to select from a set of products. Each product has a unique product code, description, and price tag. While description is not talked about, it is a common-sense attribute to make the product model more realistic.
- **Inventory management:** Products are stored in specific racks within the vending machine. The system keeps track of the inventory level for each product in its respective rack.
- **Payment processing:** The system only accepts cash payments and can calculate change when needed.

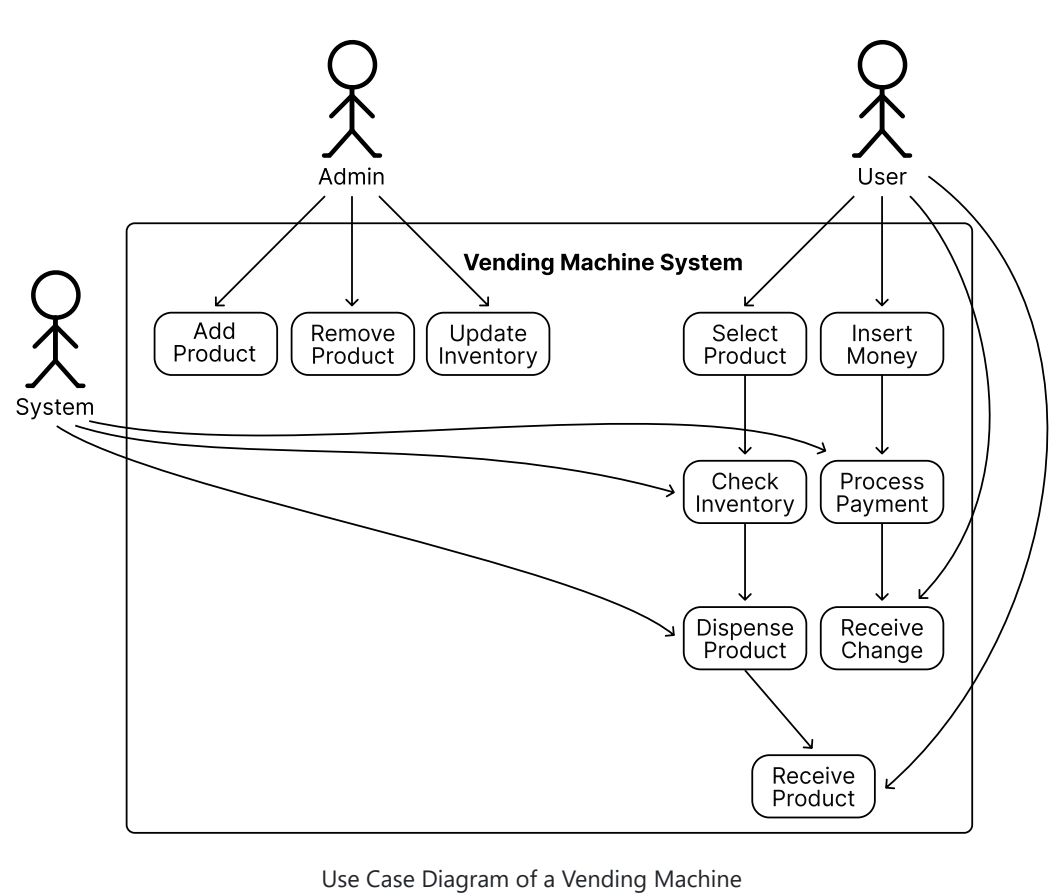
Below are the non-functional requirements:

- The user interface must be intuitive, allowing users to complete a purchase (insert money, select product, receive product, and change) with minimal instructions, and error messages should be clear and concise to guide users effectively.
- The system must protect against unauthorized access to the vending machine, ensuring only admins can add, remove, or update products, and securely handle cash transactions to prevent tampering or fraud.

Use Case Diagram

A use case diagram illustrates how actors (users or the system) interact with the vending machine system to achieve specific goals. This diagram helps clarify key actions, such as inserting money, selecting a product, dispensing items, and managing inventory.

Below is the use case diagram of the vending machine system.



Use Case Diagram of a Vending Machine

The use cases for the **User** actor are as follows:

- Insert Money: The user inserts cash to initiate a purchase.
- Select Product: The user chooses a product by entering its unique product code.
- Receive Product: The user collects the dispensed product from the vending machine.
- Receive Change: The user receives any change if the inserted amount exceeds the product's price.

The use cases for the **Admin** actor are as follows:

- Add Product: The admin adds new products to the vending machine's inventory.
- Remove Product: The admin removes products from the vending machine's inventory.
- Update Inventory: The admin updates the stock levels of existing products in the racks.

The use cases for the **System** actor are as follows.

- Process Payment: The system validates the inserted cash and calculates change if necessary.
- Dispense Product: The system releases the selected product from the appropriate rack.
- Check Inventory: The system verifies the availability of a product before dispensing.
- Display Message: The system shows messages or errors to guide the user (e.g., "Insert money to proceed," "Product out of stock," or "Insufficient funds").

Identify Core Objects

Before diving into the design, it's important to enumerate the core objects and give them appropriate names. These objects will form the foundation of the vending machine's structure and functionality.

- **VendingMachine:** This is the central entity that coordinates the vending machine's operations and serves as the main entry point for user interactions. We will ensure this entity does not become a god object (antipattern) by appropriately delegating responsibilities to other components. The Facade design pattern is very helpful in this case, as it allows a single class to orchestrate end-to-end functionality.
- **Product:** Represents the items stored in the vending machine. Each product has attributes like an identifier, a price, and a description. Products are also linked to the racks where they are stored.
- **Rack:** Represents a designated slot in the vending machine that holds a single product type and stores multiple units of that product. It also includes the dispenser hardware to release one unit of a product at a time upon selection.

> **Design choice:** Products are linked to racks because racks represent the physical storage locations, and products are associated with racks since they are static entities that don't manage their storage. This aligns with the single responsibility principle. Alternatively, racks could be linked to products, but this would violate the single responsibility principle, as racks need to manage product details.

- **InventoryManager:** Keeps track of the inventory level within the vending machine.
- **PaymentProcessor:** Interacts with coin dispensers to process payment. It also keeps track of the vending machine balance and calculates change when needed.

Design Class Diagram

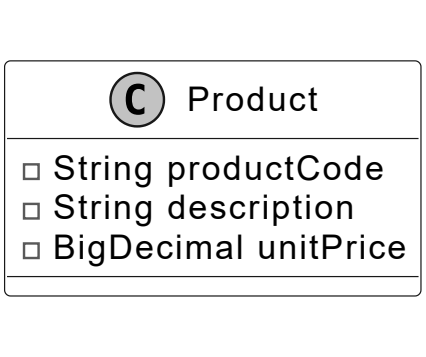
Now that we know the core objects and their roles, the next step is to create classes and methods to build the vending machine system.

Product

The first component in our class diagram is the Product class, which represents a basic product within the vending machine. It includes attributes such as product code, description, and the price of the product.

While additional attributes could be added for completeness, we skip them for this exercise. During the interview, it's a good idea to acknowledge these other attributes but focus on the essential attributes to save time and stay aligned with requirements.

Below is the representation of this class.



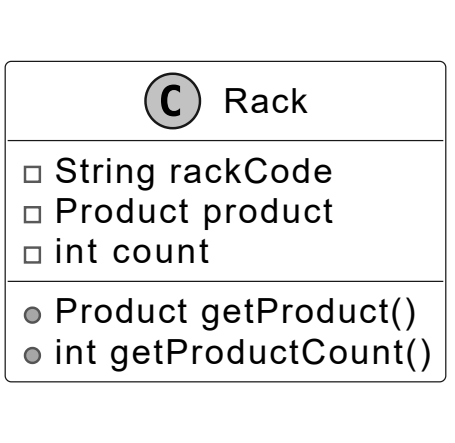
> **Design choice:** One thing to note is that we have not modeled the inventory quantity within the product class. The product class encapsulates innate properties like its code, description, and price. The stock level of our vending machine racks is constantly changing. Recognizing this distinction supports cleaner object decomposition and adherence to the single responsibility principle. We introduce a separate InventoryManager class to manage stock levels.

Rack

Next, we will look at the Rack class, which models a single rack space within the vending machine. Each rack is associated with a single product and can hold multiple units of that product.

We will put multiple racks together via the *composition* technique to represent the inventory spaces within the vending machine.

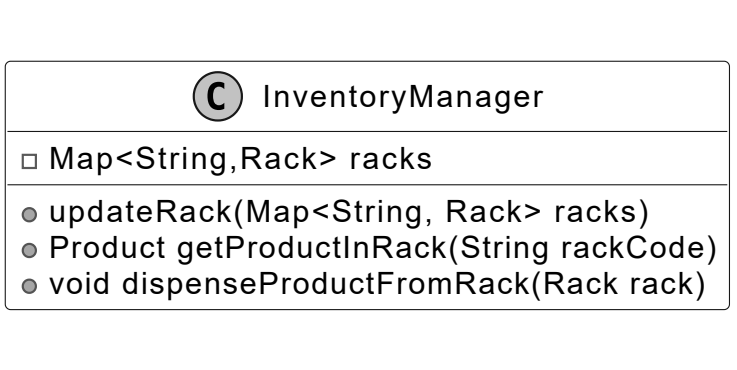
Here is the representation of this class.



> **Design choice:** We chose not to have the Rack class include methods like dispenseProductFromRack. Instead, we kept the Rack class focused on managing inventory count and product information, delegating actions like dispensing to a higher-level class, such as InventoryManager, which aligns with the single responsibility principle.

InventoryManager

Building on the Rack class, the InventoryManager class handles the tracking and storage of products in the vending machine. It supports operations such as adding, removing, and dispensing products during user interactions. It will interface with hardware mechanisms that dispense items from the rack.


C InventoryManager
Map<String,Rack> racks
updateRack(Map<String, Rack> racks)
Product getProductInRack(String rackCode)
void dispenseProductFromRack(Rack rack)

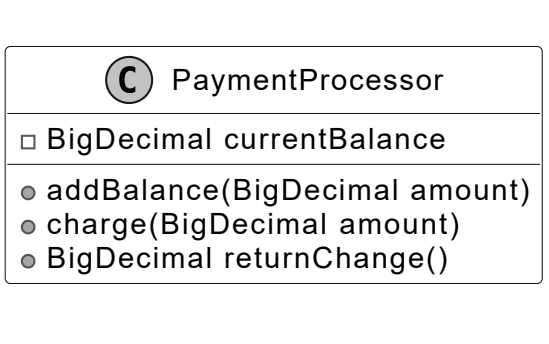
Key method: The dispenseProductFromRack method executes the action of dispensing product from the rack and decrements the inventory level. Pay attention to the naming of dispenseProductFromRack and getProductInRack. To avoid ambiguity, we should follow conventions and reserve the "get" prefix for getters that return attributes.

The updateRack method allows for editing product offerings or inventory levels. This allows an admin to edit the state of the rack.

> **Design choice:** When managing collections like racks in InventoryManager, we must decide whether to expose the collection directly, a copy, or specific methods. The choice should balance flexibility and control. Here, we use updateRack(Map racks) to allow administrative components to replace the entire rack structure in one operation, suitable for bulk updates. For most cases, we prefer granular methods like addRack(Rack rack) and removeRack(Rack rack) to limit modifications to individual racks, reducing the risk of unintended changes. These methods are used for read access, aligning with the vending machine's needs. To enhance safety, consider immutable collections or defensive copying to prevent unintended modifications and ensure thread safety in multi-threaded environments.

PaymentProcessor

With inventory management addressed, we now turn to the PaymentProcessor class. This class manages payment acceptance, including tracking the current balance and returning change. This will interface with a coin receptacle or a credit card processing unit if supported.


C PaymentProcessor
BigDecimal currentBalance
addBalance(BigDecimal amount)
charge(BigDecimal amount)
BigDecimal returnChange()

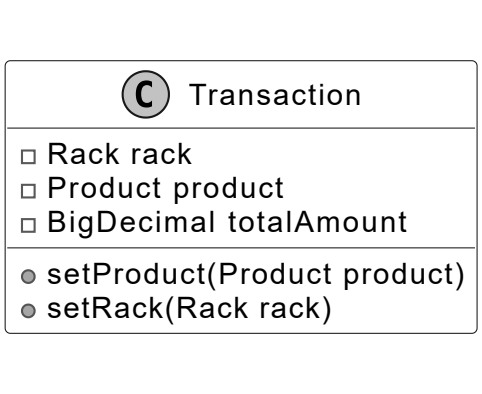
Transaction

In a vending machine system, purchases involve multiple steps, including product selection, payment processing, and confirmation. While components like PaymentProcessor handle payments and InventoryManager manage stock, the Transaction class acts as a data structure that tracks the current state of a purchase.

This design provides several benefits.

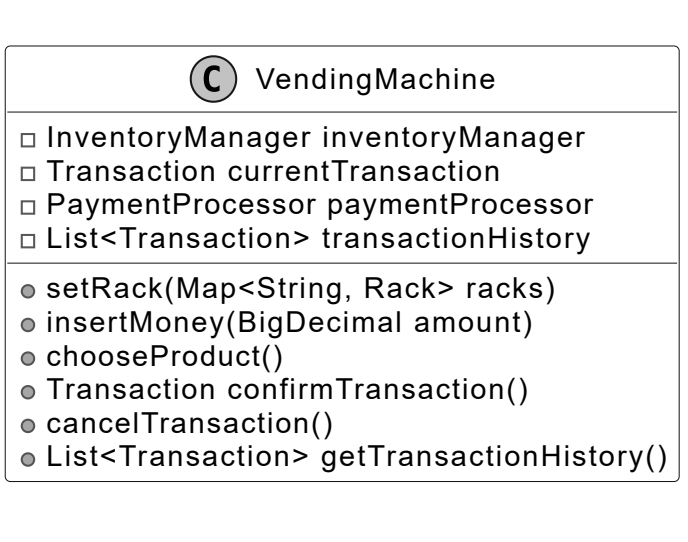
- It encapsulates key details such as the selected product, the rack it belongs to, and the total cost required for the purchase.
- By maintaining a structured record, the vending machine can track an in-progress transaction before finalizing or canceling it.
- The Transaction class improves coordination between different components. While PaymentProcessor is responsible for deducting the required amount, and InventoryManager ensures the selected product is available and dispenses it, the Transaction object ensures that the vending machine keeps all necessary purchase details in one place so that the system can reference them throughout the transaction process.

Below is the representation of this class.



VendingMachine

This class serves as the core component of the system. Here is the representation of the class:


C VendingMachine
InventoryManager inventoryManager
Transaction currentTransaction
PaymentProcessor paymentProcessor
List<Transaction> transactionHistory
setRack(Map<String, Rack> racks)
insertMoney(BigDecimal amount)
chooseProduct()
Transaction confirmTransaction()
cancelTransaction()
List<Transaction> getTransactionHistory()

The VendingMachine class models a vending machine's behavior, processes payments, and manages inventory.

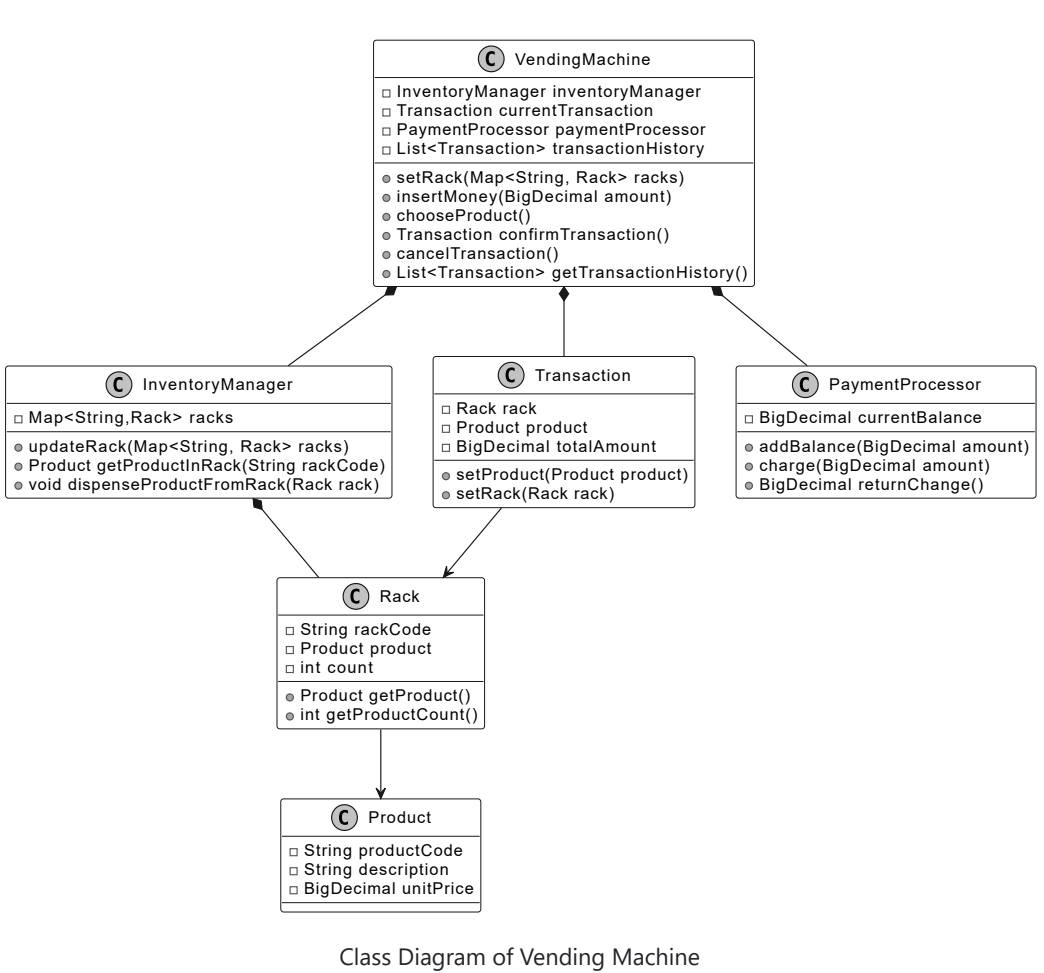
Design pattern: The Vending Machine uses the *Facade* pattern to provide a single interface to the clients of the Vending Machine. The term client refers to the software or hardware interfaces of the vending machine rather than any individual users.

Note: To learn more about the Facade pattern and its common use cases, refer to the **Parking Lot** chapter of this chapter.

> **Design choice:** To prevent the VendingMachine class from becoming a "*god object*" (a class with too many responsibilities), facades should remain lightweight and delegate tasks to other classes that adhere to the single responsibility principle. For example, the vending machine delegates product management to the InventoryManager and payment handling to the PaymentProcessor.

Complete Class Diagram

Below is the complete class diagram of our vending machine system:


C VendingMachine
InventoryManager inventoryManager
Transaction currentTransaction
PaymentProcessor paymentProcessor
List<Transaction> transactionHistory
setRack(Map<String, Rack> racks)
insertMoney(BigDecimal amount)
chooseProduct()
Transaction confirmTransaction()
cancelTransaction()
List<Transaction> getTransactionHistory()
C InventoryManager
Map<String,Rack> racks
updateRack(Map<String, Rack> racks)
Product getProductInRack(String rackCode)
void dispenseProductFromRack(Rack rack)
C Transaction
Rack rack
Product product
BigDecimal totalAmount
setProduct(Product product)
setRack(Rack rack)
C PaymentProcessor
BigDecimal currentBalance
addBalance(BigDecimal amount)
charge(BigDecimal amount)
BigDecimal returnChange()
C Rack
String rackCode
Product product
int count
Product getProduct()
int getProductCount()
C Product
String productCode
String description
BigDecimal unitPrice

Class Diagram of Vending Machine

Code - Vending Machine

In this section, we'll implement the core functionalities of the vending machine system, focusing on key areas such as managing product inventory, processing cash payments, and handling product selection and dispensing.

Product

We will start by implementing the Product class, which represents a basic unit of a product in the context of a vending machine. The definition of the Product class is given below:

```java
class Product {
    final String productCode;
    final String description;
    final BigDecimal unitPrice;

    public Product(String productCode, String description, BigDecimal unitPrice) {
        this.productCode = productCode;
        this.description = description;
        this.unitPrice = unitPrice;
    }
}
```

Implementation choice: For monetary values like the unitPrice attribute, we recommend using BigDecimal for its precision and rounding control. For an interview, it is also acceptable to use an integer to represent the smallest unit of currency (e.g., cents for US dollars) to save time. Avoid using float or double for currency, as they introduce precision/rounding errors. For identifiers like productCode, we recommend using a string rather than a numeric type in your code, even if the values are digits, you will most likely not be performing calculations, but string operations.

InventoryManager and Rack

Next, we implement the InventoryManager and Rack classes, which work together to manage the vending machine's inventory. By using the Composite design pattern, we create a hierarchical structure for handling inventory at multiple levels. The InventoryManager class manages the overall inventory, while the Rack class handles individual storage units.

We use a HashMap<String, Rack> to store racks because it allows for efficient lookups by rack code. Since each rack has a unique identifier, a hash map provides constant-time (O(1)) access when retrieving or updating a rack. This makes it well-suited for managing inventory in a vending machine, where quick access to product storage is important.

Below is the representation of the two classes:

```java
public class InventoryManager {
    // Maps rack codes to their corresponding rack objects
    private Map<String, Rack> racks;

    public InventoryManager() {
        racks = new HashMap<>();
    }

    // Retrieves the product from a specific rack using its code
    public Product getProductInRack(String rackCode) {
        return racks.get(rackCode).getProduct();
    }

    // Dispenses a product from the specified rack and decrements its count
    public void dispenseProductFromRack(Rack rack) {
        if (rack.getProductCount() > 0) {
            rack.setCount(rack.getProductCount() - 1);
        } else {
            throw new IllegalStateException("Cannot dispense product. Rack is empty.");
        }
    }

    public void updateRack(Map<String, Rack> racks) {
        this.racks = racks;
    }

    public Rack getRack(String name) {
        return racks.get(name);
    }
}
```

Rack class

The Rack class represents individual storage units in the vending machine, each associated with a single product type.

```java
public class Rack {
    private final String rackCode;
    private final Product product;
    private int count;

    public Rack(final String rackCode, final Product product, final int count) {
        this.rackCode = rackCode;
        this.product = product;
        this.count = count;
    }

    public Product getProduct() {
        return product;
    }

    public int getProductCount() {
        return count;
    }
}
```

PaymentProcessor

The PaymentProcessor class handles payment-related operations, such as adding funds, charging for purchases, and returning change. This ensures the vending machine's financial logic is encapsulated and easily maintainable.

```java
public class PaymentProcessor {
    // Tracks the current balance in the payment processor
    private BigDecimal currentBalance = BigDecimal.ZERO;

    // Adds the specified amount to the current balance
    public void addBalance(BigDecimal amount) {
        currentBalance = currentBalance.add(amount);
    }

    // Deducts the specified amount from the current balance
    public void charge(BigDecimal amount) {
        currentBalance = currentBalance.subtract(amount);
    }

    // Returns the current balance as change and resets the balance to zero
    public BigDecimal returnChange() {
        BigDecimal change = currentBalance;
        currentBalance = BigDecimal.ZERO;
        return change;
    }

    // Returns the current balance
    public BigDecimal getCurrentBalance() {
        return currentBalance;
    }
}
```

VendingMachine

Finally, we implement the VendingMachine class, a central component in the vending machine that is responsible for modeling the vending machine's behavior and handling user interactions.

Below is the implementation of this class.

```java
class VendingMachine {
    // Stores the history of all completed transactions
    private final List<Transaction> transactionHistory;
    // Manages the inventory of products in the vending machine
    private final InventoryManager inventoryManager;
    // Handles all payment-related operations
    private final PaymentProcessor paymentProcessor;

    // Tracks the current ongoing transaction
    private Transaction currentTransaction;
    // Represents the current state of the vending machine
    private VendingMachineState currentState;
    // Tracks the current balance in the machine
    private double balance;
    // Stores the currently selected product code
    private String selectedProduct;

    public VendingMachine() {
        transactionHistory = new ArrayList<>();
        currentTransaction = new Transaction();
        inventoryManager = new InventoryManager();
        paymentProcessor = new PaymentProcessor();
        this.currentState = new NoMoneyInsertedState();
        this.balance = 0.0;
        this.selectedProduct = null;
    }

    // Updates the rack configuration with new product racks
    void setRack(Map<String, Rack> rack) {
        inventoryManager.updateRack(rack);
    }

    // Adds money to the payment processor
    void insertMoney(final BigDecimal amount) {
        paymentProcessor.addBalance(amount);
    }

    // Selects a product from a specific rack
    void chooseProduct(String rackId) {
        final Product product = inventoryManager.getProductInRack(rackId);
        currentTransaction.setRack(inventoryManager.getRack(rackId));
        currentTransaction.setProduct(product);
    }

    // Processes and completes the current transaction
    Transaction confirmTransaction() throws InvalidTransactionException {
        // Step 1: Validate the transaction before processing
        validateTransaction();

        // Step 2: Charge the customer for the product
        paymentProcessor.charge(currentTransaction.getProduct().getUnitPrice());

        // Step 3: Dispense the product from the rack
        inventoryManager.dispenseProductFromRack(currentTransaction.getRack());

        // Step 4: Return the change to the customer
        currentTransaction.setTotalAmount(paymentProcessor.returnChange());

        // Step 5: Add the completed transaction to the history
        transactionHistory.add(currentTransaction);
        Transaction completedTransaction = currentTransaction;

        // Reset the current transaction for the next purchase.
        currentTransaction = new Transaction();
        return completedTransaction;
    }

    // Validates the current transaction for product availability and sufficient funds
    private void validateTransaction() throws InvalidTransactionException {
        if (currentTransaction.getProduct() == null) {
            throw new InvalidTransactionException("Invalid product selection");
        } else if (currentTransaction.getRack().getProductCount() == 0) {
            throw new InvalidTransactionException("Insufficient inventory for product.");
        } else if (paymentProcessor
                        .getCurrentBalance()
                        .compareTo(currentTransaction.getProduct().getUnitPrice())
                < 0) {
            throw new InvalidTransactionException("Insufficient fund");
        }
    }

    // Returns an unmodifiable list of all completed transactions
    public List<Transaction> getTransactionHistory() {
        return Collections.unmodifiableList(transactionHistory);
    }

    // Cancels the current transaction and returns any inserted money
    public void cancelTransaction() {
        paymentProcessor.returnChange();
        currentTransaction =
                new Transaction(); // Reset the current transaction for the next purchase.
    }

    // Returns the inventory manager instance
    public InventoryManager getInventoryManager() {
        return inventoryManager;
    }
}
```

Let's walk through the purchase process and highlight the methods' roles.

- insertMoney(BigDecimal amount): Adds the specified amount to the machine's balance via the `PaymentProcessor` class.
- chooseProduct(): Retrieves the product and its corresponding rack from the InventoryManager and associates them with the current transaction.
- confirmTransaction(): Validates the transaction (e.g., checks for sufficient funds and product availability), processes payment, dispenses the product, and updates the machine's inventory and transaction history.

Deep Dive Topics

Now that the basic design is complete, the interviewer might ask you to enhance the vending machine's functionality or accommodate more complex use cases.

Enforcing task sequences

What if the interviewer asks: "How would you ensure that users insert money before selecting a product?" This is a common requirement in vending machines to prevent invalid actions, such as selecting a product without committing to payment.

To address this, we need to enforce a strict sequence of actions:

1. Users must insert money first.
2. The system checks if the inserted amount is enough for a purchase.
3. If the amount is sufficient, the user can select a product.
4. Finally, the machine dispenses the product.

Additionally, the vending machine should provide feedback at each stage to guide the user. For instance, it might display messages like "Insert money to proceed," "Select a product," or "Please collect your change." How would you go about implementing this?

To handle these requirements, we can introduce the State Pattern. This pattern allows us to model the vending machine's behavior as a set of well-defined states. Let's break it down.

Note: To learn more about the State Pattern and its common use cases, refer to the **Further Reading** section at the end of this chapter.

Design changes

To enforce task sequences and display state-dependent messages, we will define three distinct states:

NoMoneyInsertedState:

- Represents the initial state where no money has been inserted.
- Displays prompts like *"Insert money to proceed."*
- Users are only allowed to insert money. If a user attempts to select a product without inserting money, it raises an exception.
- Transitions to MoneyInsertedState upon successful money insertion.

MoneyInsertedState:

- Represents the state where money has been inserted.
- Displays prompts like *"Select a product."*
- Enables product selection while preventing additional money insertion to avoid overpayment.
- Validates if the selected product is available and the inserted amount covers the product's cost.
- Transitions to DispenseState upon successful product selection.

DispenseState:

- Represents the state where the vending machine is prepared to dispense the selected product.
- Displays prompts like *"Dispensing product…"* or *"Please collect your change."*
- Handles product dispensing and resets the machine to the initial state after completion.
- Prevents further actions (e.g., inserting money or selecting another product) until the process is complete.

Why does this work?

The State Pattern explicitly defines the transitions between states, ensuring that actions follow the required order. Here's how it works:

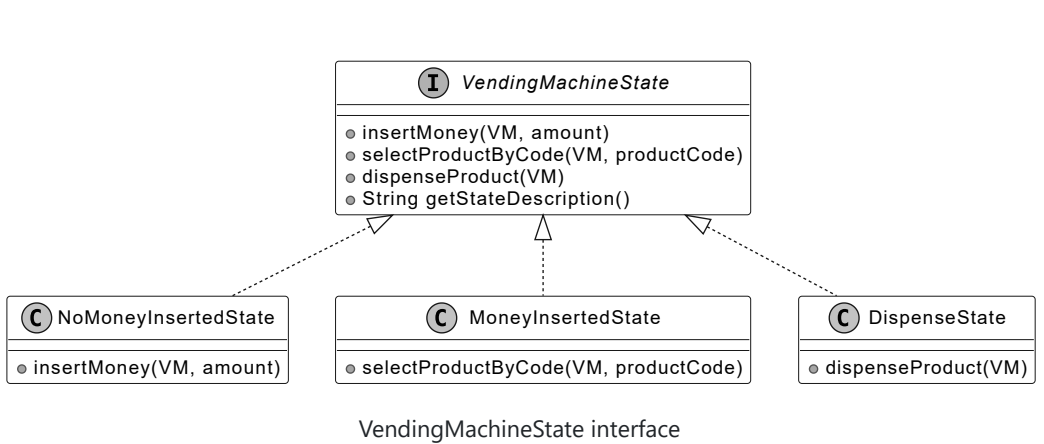
- In NoMoneyInsertedState, users can only insert money. If they try to select a product, the system raises an error.
- In MoneyInsertedState, users must select a product before the system dispenses anything.
- In DispenseState, the vending machine completes the transaction and prevents further actions until it resets.

This approach guarantees the sequence: **Insert Money → Select Product → Dispense Product**.

Each state provides user feedback based on its context:

- NoMoneyInsertedState: "Insert money to proceed."
- MoneyInsertedState: "Select a product."
- DispenseState: "Dispensing product…" or "Please collect your change."

We now define a VendingMachineState interface that serves as a blueprint for the three states (NoMoneyInsertedState, MoneyInsertedState, and DispenseState).



VendingMachineState interface

Code changes

The VendingMachineState interface sets the rules for all states of the vending machine. It includes the behaviors that different states of the vending machine should implement, such as inserting money, selecting products, dispensing products, and describing the current state.

```java
public interface VendingMachineState {
    // Handles money insertion in the current state
    void insertMoney(VendingMachine VM, double amount);

    // Handles product selection in the current state
    void selectProductByCode(VendingMachine VM, String productCode)
            throws InvalidStateException;

    // Handles product dispensing in the current state
    void dispenseProduct(VendingMachine VM) throws InvalidStateException;

    // Returns a description of the current state
    String getStateDescription();
}
```

Here is the code for the NoMoneyInsertedState class:

```java
public class NoMoneyInsertedState implements VendingMachineState {
    // Adds money to the machine and transitions to MoneyInsertedState
    @Override
    public void insertMoney(VendingMachine VM, double amount) {
        VM.addBalance(amount);
        VM.setState(new MoneyInsertedState());
    }

    // Throws exception as product selection is not allowed without money
    @Override
    public void selectProductByCode(VendingMachine VM, String productCode)
            throws InvalidStateException {
        throw new InvalidStateException("Cannot select a product without inserting money.");
    }

    // Throws exception as product dispensing is not allowed without money
    @Override
    public void dispenseProduct(VendingMachine VM) throws InvalidStateException {
        throw new InvalidStateException("Cannot dispense product without inserting money.");
    }

    // Returns a description of the current state
    @Override
    public String getStateDescription() {
        return "No Money Inserted State - Please insert money to proceed";
    }
}
```

For brevity, the implementation of the MoneyInsertedState and DispenseState classes is omitted, but they follow the same structure.

Wrap Up

In this chapter, we have designed and implemented a Vending Machine system. The most important takeaway from this chapter is how we divided responsibilities across classes, such as Product, Rack, InventoryManager, and PaymentProcessor, while unifying them under a facade for a clear and simple-to-access API. This approach not only simplified the system's external interface but also adhered to the Single Responsibility Principle, ensuring each component focused on a specific responsibility. For instance, the InventoryManager managed stock levels, while the PaymentProcessor handled cash payments and calculated change.

In the deep dive section, we explored state-based control using the State Pattern to enforce a strict sequence of actions and prevent invalid behaviors like dispensing without payment.

In interviews, remember to emphasize validation and error handling after implementing core functionality, especially for systems where improper behavior could cause damage or financial loss.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Further Reading: State Design Pattern

This section gives a quick overview of the design patterns used in this chapter. It's helpful if you're new to these patterns or need a refresher to better understand the design choices.

State design pattern

The State pattern is a behavioral pattern that allows an object to alter its behavior when its internal state changes, making it appear as though the object is behaving like a different class.

In the vending machine design, we use the State pattern to manage states like NoMoneyInsertedState, MoneyInsertedState, and DispenseState, enabling the VendingMachine to switch behaviors dynamically without modifying its core logic.

To illustrate the State pattern in another domain, the following example uses the traffic light system.

Problem

Imagine we have a TrafficLight class. The traffic light can be in one of three states: Red, Yellow, or Green. The behavior of the traffic light changes depending on its current state:

- In the Red state, the light stays red for a set duration.
- In the Yellow state, the light blinks yellow, signaling cars to slow down and prepare to stop.
- In the Green state, the light stays green to allow traffic to pass.

If we were to implement this logic using conditionals, we would need to check the current state of the traffic light every time an action occurs.

While the solution works initially, several issues arise as the system becomes more complex:

Scalability: As the number of states increases, the conditionals grow larger. For example, adding a new state (like a flashing state for emergency vehicles) would require adding more checks to the existing logic, making the code increasingly hard to manage and prone to errors.

Maintainability: The duplication of code and the need to update the same conditional logic in multiple places make the system difficult to maintain over time. This is a crucial problem because it impacts long-term code quality and increases the chance of introducing bugs when modifying the logic.

Solution

Instead of relying on conditionals to manage state transitions, we can use the State pattern, which encapsulates the behavior associated with each state into separate classes.

Rather than handling all behaviors on its own, the original object, known as the context, holds a reference to one of the state objects that represents its current state, delegating the state-related tasks to that object.

For example, a TrafficLight context can delegate its behavior to a state object, like RedLightState, GreenLightState, or YellowLightState. Each of these states knows how to handle the actions specific to that state, such as changing the light or transitioning to the next state.

To transition to a new state, the context simply replaces the current state object with another one that represents the new state. For instance, when the light is Green, the system transitions to Yellow, and then to Red, without needing complex conditionals.

Here is the representation of the state pattern.

State design pattern

☐ Mark as Complete